

JavaScript mid-term exam (2020-21-1)

Due No Due Date **Points** None

Important information

- You have **120 minutes** to complete the exam. We provide you **additional 30 minutes** for filling in the `README.md` (see template below), reading the problems, downloading the materials, compressing and uploading your solution.
- Solutions must be uploaded to ELTE Canvas. **The system closes at 18:30 (Hungarian time) and there will be no opportunity to hand in your solution after the aforementioned timepoint.**
- You may use **any kind of material** (including documentation, lectures, class exercises, cheat sheets, etc) during the exam, **except human help** (synchronous, asynchronous, chat, forums, etc)!
- The problems are independent and can be solved **in any order**.
- To begin your work you should first [download the starting package we have prepared](#). Each task has its own directory in the package. We have already prepared the HTML, CSS and JavaScript files in each directory. Use these to complete your work. You only have to modify the `.js` files but you are allowed to edit the HTML if needed. You may also use multiple `.js` files per solution but this is not mandatory.
- The starting package contains a template called `README.md` where you should fill in your name and Neptun code (the `<>` characters must be removed). **Submissions without a properly filled `README.md` file will not be graded!**

```
<Name>
<Neptun ID>
This solution was submitted and prepared by the student stated above for the assignment of the Web programming course.
I declare that this solution is my own work.
I have not copied or used third party solutions.
I have not passed my solution to my classmates, neither made it public.
Students' regulation of Eötvös Loránd University (ELTE Regulations Vol. II. 74/C. § ) states
that as long as a student presents another student's work - or at least the significant part of it -
as his/her own performance, it will count as a disciplinary fault. The most serious consequence of a
disciplinary fault can be dismissal of the student from the University.
```

- Each directory contains a file named `TASKS.md`. Replace the space in `[ ]` with an `x` where you have (even partially) succeeded to complete the task. This will make us easier to find your solution when we grade your exam.

Problem 1: Haiku editor (haiku)

Create a haiku editor. Haiku is a Japanese form of poetry where a poem consists of 3 lines and the lines contain 5-7-5 syllables. In this problem we are going to assume that the number of syllables in a word matches the number of vowels it contains. (Note: this assumption is actually true for Hungarian words, but extremely untrue for English.) The haiku editor is a simple multi-line text input (`textarea`) where we can

write our poem and begin a new line by pressing ENTER. While typing, the editor displays various statistics about the entered text; and if the text matches the requirements for a haiku then the editor becomes green and a button appears which appends the haiku to a list. Tasks:

- a. The file `index.html` already contains all HTML elements you will need to solve the problem. Each element has some initial content which has to be overwritten by your solution but at least you can see the format of the expected output.
- b. (1 pt) Output the contents of the field to the console while typing using an `input` event.
- c. (1 pt) Display the number of characters in the editor to the HTML element with the ID `number-of-characters`.
- d. (1 pt) Display the number of rows in the editor to the HTML element with ID `number-of-rows`. (Rows are separated by `\n`)
- e. (1 pt) Output the number of vowels in the first row to the console while typing. The Hungarian vowels are: `aáeeíioóöőuúüű`.
- f. (1.5 pts) List the number of vowels per row in the HTML list with ID `vowels-per-row`.
- g. (1.5 pts) If the number of rows is 3 and they consist of 5-7-5 syllables then add the style class `haiku` to the `p` element that contains the input field. This will make the background green and a button should appear. Remove the class if the haiku conditions aren't met.
- h. (1 pt) If you apply the style correctly, a button appears. When the button is clicked you need to add the content of the editor to the `<pre>` elements inside the element with the ID `haikus`. (See examples.)

Haiku editor

Write your poem here:

```
Web programming is
```

Number of characters: 19

Number of rows: 2

Number of vowels per row:

1. 5

2. 0

Haikus

```
hűs hajnal ölel
már halkulnak a tücskök
légy türellemmel

Fölhívtál reggel...
azóta a mosolyod
bennem mosolyog...

nézlek én Téged
ahogy az ágyon fekszel
szerelmes vagyok
```

Problem 2: Bird watch (bird-watch)

We have conducted a bird watch in a some settlements across the country. We know how many birds of certain species we have observed in each settlement. Calculations should be performed in a table where cell `i,j` contains the number of birds observed in settlement `i` of species `j`. If we observe an additional one then we increase the number in the cell by clicking into the cell. Every task must be recalculated when a value in the table is updated. Attention! There is no need to know the names of settlements or bird species - we refer to them by their row or column index.

- a. The file `index.html` already contains a form where you can enter the number of settlements (`n`) and bird species (`m`). The "Generate table" button creates a matrix of size `nxm` in the background. The code to accomplish this is already done for you in file `index.js`. You need to expand the file `index.js` to complete the following tasks.
- b. (1.5 pts) Using JavaScript display the matrix stored in variable `matrix` to the HTML element `table-container` as a table (using `table`, `tr`, `td` elements) when the "Generate table" button is clicked. Eg: For `n=2` and `m=2` you should display something like:

```
<table>
  <tr>
    <td>0</td>
    <td>0</td>
  </tr>
  <tr>
    <td>0</td>
    <td>0</td>
  </tr>
</table>
```

- c. (1 pts) When a cell is clicked, output its `row,column` coordinates to the console. Use delegating if you can (see task d.).
- d. (1.5 pts) Solve task c. by registering only one event listener (delegating).
- e. (1 pts) The following tasks must be recalculated and redisplayed after every cell click. It might be helpful and timesaving the use array functions as it takes only one row to solve each task with them. If you couldn't solve tasks b. or c. then you may use a hardcoded matrix to continue.
- f. (1 pts) Display (using words "Yes" or "No") whether any bird has been observed in the first settlement (whether the first row of the matrix contains any value other than zero) to the element with ID `task-1`.
- g. (1 pts) Display the number of settlements where more than 10 birds of the same species have been observed (how many rows contain at least one value greater than 10) to the element with ID `task-2`.
- h. (1 pts) Display the row index (starting from 1 (ONE)) of a settlement where no bird has been observed at all (the whole row only contains zeros) to the element with ID `task-3`. Display the text "No" if no such settlement exists.

Bird watch

Places:

Species:

Bird counts

0	0	0	0
0	0	0	0
0	0	0	0

Questions

Is there any bird in the first place?

No

In how many places have they seen more than 10 birds from any of the bird species?

0

Give one place where no birds have been seen! (row number indexed from 1, "No" if there is no such place)

1

Problem 3: Audio visualisation (canvas-audio)

Audio is sometimes visualised by displaying the intensity of various frequencies, for example by connected line segments. There will be no real audio in this problem but we will simulate an audio-like data and create a visualiser using 2D bitmap graphics.

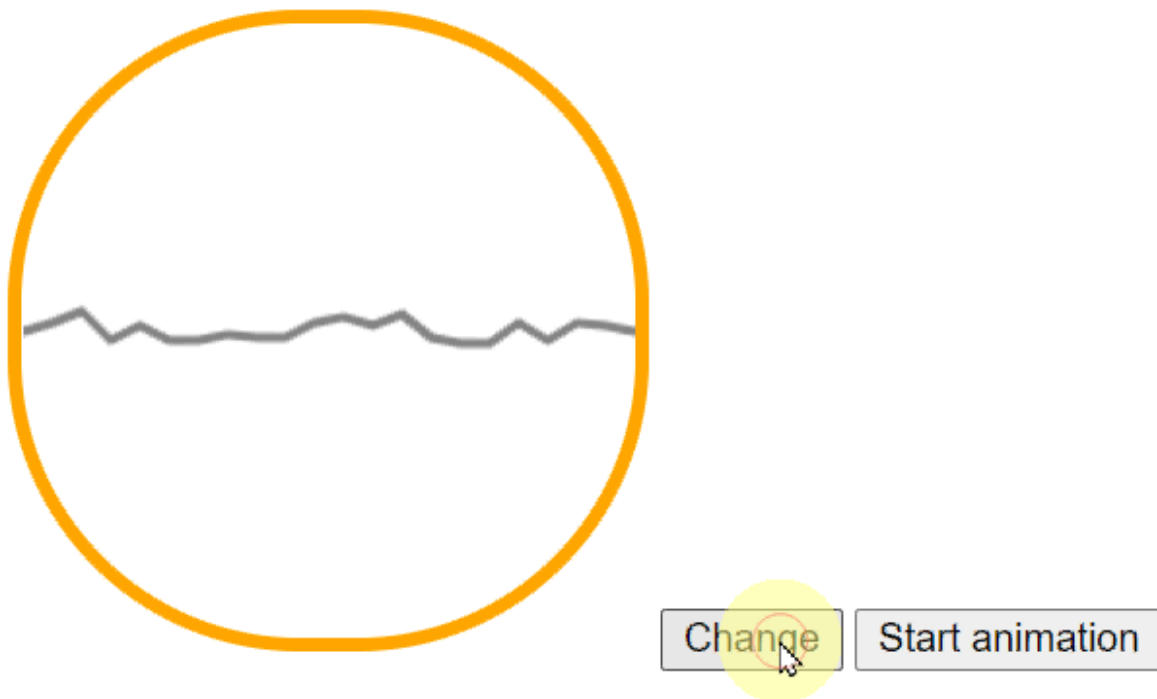
- a. The file `index.html` already contains a 210 x 210 `canvas` element, some styling and two buttons for the tasks. You need to complete the tasks in the supplied empty `index.js` file.
- b. (1 pt) Create the function `random(a, b)` that returns a random integer from the closed interval $[a, b]$. Output a random integer between -10 and 10 (inclusive) to the console.
- c. (1.5 pt) c. Fill an array of length 20 with random values between -5 and 5 (inclusive) and output the filled array to the console.
- d. (1.5 pt) Draw the elements of the array to the canvas following these rules. Draw a broken line from the vertical center of the left edge to the center of the right edge so that the first line segment ends 10px to the right and at the height which corresponds to the first element of the array. The next line segment ends after an additional 10px to the right and above or below based on the second element. For example if your array contains `[-4, 1, -5, 5, ..., -3]` then the coordinates should be:

Array element X coord. Y coord. Y coord. reason

	0	105	fixed endpoint
-4	10	101	105-4
1	20	106	105+1
-5	30	100	105-5
5	40	110	105+5
...	...	...	...

-3	200	102	105-3
	210	105	fixed endpoint

- e. (1 pt) Your lines should be 3px wide and gray.
- f. (1.5 pt) When the button "Change" is clicked, add a random integer between -1 and 1 (inclusive) to every element of the array and redraw the new state to the canvas (after clearing it first).
- g. (1.5 pt) When the button "Start animation" is clicked, make the change automatic and continuous. Keep changing the elements of the array and redrawing the state to the canvas using some kind of timer mechanism (`requestAnimationFrame` [_\(https://developer.mozilla.org/en-US/docs/Web/API/window/requestAnimationFrame\)](https://developer.mozilla.org/en-US/docs/Web/API/window/requestAnimationFrame) or `setInterval` [_\(https://developer.mozilla.org/en-US/docs/Web/API/WindowOrWorkerGlobalScope/setInterval\)](https://developer.mozilla.org/en-US/docs/Web/API/WindowOrWorkerGlobalScope/setInterval)).



Problem 4: Progress manager (progress-manager)

Independent tasks of a bigger problem (eg. writing chapters of a book) have been decomposed into smaller steps. These steps have to be completed in the given order. If a step is done then we can mark it as completed by clicking on it. However sometimes we forget to keep track of our progress right away so our system has to work the following way: when somebody marks a step as completed, all other steps of that task which come before it must also be marked as completed.

- a. The file `index.html` already contains the decomposed steps of a problem. Solve the tasks in the file `index.js`.
- b. (1 pt) When a step is clicked add the style class `done` to it. Use delegation if you can (see task h.).
- c. (1 pt) Do not apply the style class `done` to tasks when they are clicked.
- d. (1.5 pt) If a step is clicked all uncompleted steps before it also get added to style class `done`.
- e. (1 pt) If a list item that has already been marked as completed is clicked then remove the style class `done` only if none of the steps after the clicked one are completed. Help: you can

use `nextElementSibling` (<https://developer.mozilla.org/en-US/docs/Web/API/NonDocumentTypeChildNode/nextElementSibling>) method to iterate through the siblings.

- f. (1 pt) If all steps of a certain task are completed then add the style class `done` to the task itself.
- g. (1 pt) If there are uncompleted steps then the task cannot be marked as completed.
- h. (1.5 pt) Solve the above tasks by registering event listeners to one element only (delegating). Help: you can use the CSS selector `parent > child` to differentiate the tasks and steps.

Progress manager

1. Todo 1
 - ✓Step 1
 - ✓Step 2
 - ✓Step 3
 - ✓Step 4
 - Step 5
2. Todo 2
 - Step 1
 - Step 2
 - Step 3
 - Step 4
 - Step 5
 - Step 6
 - Step 7
3. Todo 3
 - Step 1
 - Step 2
 - Step 3
 - Step 4
 - Step 5
 - Step 6
4. Todo 4
 - Step 1
 - Step 2
 - Step 3

JavaScript mid-term exam

Criteria	Ratings		Pts
1/b. Output the contents of the field to the console while typing using an input event.	1.0 pts Teljes megoldás	0.0 pts Nem értékelhető	1.0 pts
1/c. Display the number of characters in the editor to the HTML element with the ID number-of-characters.	1.0 pts Teljes megoldás	0.0 pts Nem értékelhető	1.0 pts
1/d.	1.0 pts	0.0 pts	

Display the number of rows in the editor to the HTML element with ID number-of-rows. (Rows are separated by \n)	Teljes megoldás	Nem értékelhető	1.0 pts
1/e. Output the number of vowels in the first row to the console while typing. The Hungarian vowels are: aáééííóóőőúúűű.	1.0 pts Teljes megoldás	0.0 pts Nem értékelhető	1.0 pts
1/f. List the number of vowels per row in the HTML list with ID vowels-per-row.	1.5 pts Teljes megoldás	0.0 pts Nem értékelhető	1.5 pts
1/g. If the number of rows is 3 and they consist of 5-7-5 syllables then add the style class haiku to the p element that contains the input field. This will make the background green and a button should appear. Remove the class if the haiku conditions aren't met.	1.5 pts Teljes megoldás	0.0 pts Nem értékelhető	1.5 pts
1/h. If you apply the style correctly, a button appears. When the button is clicked you need to add the content of the editor to the <pre> elements inside the element with the ID haikus.	1.0 pts Teljes megoldás	0.0 pts Nem értékelhető	1.0 pts
2/b. Using JavaScript display the matrix stored in variable matrix to the HTML element table-container as a table (using table, tr, td elements) when the "Generate table" button is clicked.	1.5 pts Teljes megoldás	0.0 pts Nem értékelhető	1.5 pts
2/c. When a cell is clicked, output its row,column coordinates to the console.	1.0 pts Teljes megoldás	0.0 pts Nem értékelhető	1.0 pts
2/d. Solve task c. by registering only one event listener (delegating).	1.5 pts Teljes megoldás	0.0 pts Nem értékelhető	1.5 pts
2/e. The following tasks must be recalculated and redisplayed after every cell click. It might be helpful and timesaving the use array functions as it takes only one row to solve each task with them. If you couldn't solve tasks b. or c. then you may use a hardcoded matrix to continue.	1.0 pts Teljes megoldás	0.0 pts Nem értékelhető	1.0 pts

2/f. Display (using words "Yes" or "No") whether any bird has been observed in the first settlement (whether the first row of the matrix contains any value other than zero) to the element with ID task-1.	1.0 pts Teljes megoldás	0.0 pts Nem értékelhető	1.0 pts
2/g. Display the number of settlements where more than 10 birds of the same species have been observed (how many rows contain at least one value greater than 10) to the element with ID task-2.	1.0 pts Teljes megoldás	0.0 pts Nem értékelhető	1.0 pts
2/h. Display the row index (starting from 1 (ONE)) of a settlement where no bird has been observed at all (the whole row only contains zeros) to the element with ID task-3. Display the text "No" if no such settlement exists.	1.0 pts Teljes megoldás	0.0 pts Nem értékelhető	1.0 pts
3/b. Create the function random(a, b) that returns a random integer from the closed interval [a, b]. Output a random integer between -10 and 10 (inclusive) to the console.	1.0 pts Teljes megoldás	0.0 pts Nem értékelhető	1.0 pts
3/c. Fill an array of length 20 with random values between -5 and 5 (inclusive) and output the filled array to the console.	1.5 pts Teljes megoldás	0.0 pts Nem értékelhető	1.5 pts
3/d. Draw the elements of the array to the canvas following these rules. Draw a broken line from the vertical center of the left edge to the center of the right edge so that the first line segment ends 10px to the right and at the height which corresponds to the first element of the array. The next line segment ends after an additional 10px to the right and above or below based on the second element.	1.5 pts Teljes megoldás	0.0 pts Nem értékelhető	1.5 pts
3/e. Your lines should be 3px wide and gray.	1.0 pts Teljes megoldás	0.0 pts Nem értékelhető	1.0 pts
3/f. When the button "Change" is clicked, add a random integer between -1 and 1 (inclusive) to every element of the array and redraw the new state to the canvas (after clearing it first).	1.5 pts Teljes megoldás	0.0 pts Nem értékelhető	1.5 pts
3/g.			

When the button "Start animation" is clicked, make the change automatic and continuous. Keep changing the elements of the array and redrawing the state to the canvas using some kind of timer mechanism (requestAnimationFrame or setInterval).	1.5 pts Teljes megoldás	0.0 pts Nem értékelhető	1.5 pts
4/b. When a step is clicked add the style class done to it. Use delegation if you can (see task h.).	1.0 pts Teljes megoldás	0.0 pts Nem értékelhető	1.0 pts
4/c. Do not apply the style class done to tasks when they are clicked.	1.0 pts Teljes megoldás	0.0 pts Nem értékelhető	1.0 pts
4/d. If a step is clicked all uncompleted steps before it also get added to style class done.	1.5 pts Teljes megoldás	0.0 pts Nem értékelhető	1.5 pts
4/e. If a list item that has already been marked as completed is clicked then remove the style class done only if none of the steps after the clicked one are completed. Help: you can use nextElementSibling method to iterate through the siblings.	1.0 pts Teljes megoldás	0.0 pts Nem értékelhető	1.0 pts
4/f. If all steps of a certain task are completed then add the style class done to the task itself.	1.0 pts Teljes megoldás	0.0 pts Nem értékelhető	1.0 pts
4/g. If there are uncompleted steps then the task cannot be marked as completed.	1.0 pts Teljes megoldás	0.0 pts Nem értékelhető	1.0 pts
4/h. Solve the above tasks by registering event listeners to one element only (delegating). Help: you can use the CSS selector parent > child to differentiate the tasks and steps.	1.5 pts Teljes megoldás	0.0 pts Nem értékelhető	1.5 pts
Total Points: 32.0			